



Natural Sort order

Natural sort order is a way of sorting strings or text data in a manner that is more intuitive to humans. In natural sort order, alphanumeric strings are sorted based on their numeric components rather than just their character codes.

Typically, when sorting strings using a regular lexicographic sort order, the sorting is performed based on the ASCII or Unicode values of the characters. This means that numbers within strings are treated as individual characters rather than numeric values. As a result, strings like "file1", "file10", and "file2" would be sorted in lexicographic order as "file1", "file10", and "file2", which is not the intuitive ordering.

In contrast, natural sort order takes into account the numerical values within the strings. It treats numbers as numbers rather than individual characters. Using natural sort order, the same strings would be sorted as "file1", "file2", and "file10", which is the expected and intuitive ordering.

Natural sort order can be implemented using various algorithms or techniques. One common approach involves splitting the strings into chunks of numeric and non-numeric parts, and then sorting based on these parts. This allows the numbers within the strings to be compared and sorted in a meaningful way.

Natural sort order is particularly useful when dealing with file names, version numbers, or any other alphanumeric data that contains numbers. It provides a more human-friendly and logical sorting order compared to simple lexicographic sorting.

Example

1. Consider the list of file names:

- file1.txt
- file10.txt
- file2.txt
- file20.txt
- file3.txt

2. In the first step, we need to split each file name into numeric and non-numeric parts. This will allow us to compare the numeric parts as numbers rather than individual characters.

After splitting, we have the following pairs for each file name:

- file1.txt => (file, 1, .txt)
- file10.txt => (file, 10, .txt)
- file2.txt => (file, 2, .txt)
- file20.txt => (file, 20, .txt)
- file3.txt => (file, 3, .txt)

3. Now, we can compare the numeric parts of each pair. We sort the pairs based on the numeric parts in ascending order. If the numeric parts are equal, we can compare the non-numeric parts using regular string comparison.

Sorting the pairs gives us:

- (file, 1, .txt)
- (file, 2, .txt)
- (file, 3, .txt)
- (file, 10, .txt)
- (file, 20, .txt)

4. Finally, we concatenate the sorted pairs back together to form the sorted list of file names:

- file1.txt
- file2.txt
- file3.txt
- file10.txt
- file20.txt

So, using natural sort order, the list of file names is sorted in the expected and intuitive order: "file1.txt", "file2.txt", "file3.txt", "file10.txt", and "file20.txt".

```
1. import java.util.ArrayList;
2. import java.util.Comparator;
3. import java.util.List;
4. import java.util.regex.Matcher;
5. import java.util.regex.Pattern;

6. public class NaturalSort {
7.     public static void main(String[] args) {
8.         List<String> strings = new ArrayList<>();
9.         strings.add("file1.txt");
10.        strings.add("file10.txt");
11.        strings.add("file2.txt");
12.        strings.add("file20.txt");
13.        strings.add("file3.txt");

14.        List<String> sortedStrings =
            naturalSort(strings);

15.        for (String str : sortedStrings) {
16.            System.out.println(str);
17.        }
18.    }
```

```
1.     public static List<String> naturalSort(List<String> strings) {
2.         strings.sort(Comparator.comparing(NaturalSort::extractNumericPa
            rts)
3.             .thenComparing(Comparator.naturalOrder()));
4.         return strings;
5.     }

6.     private static List<String> extractNumericParts(String str) {
7.         List<String> numericParts = new ArrayList<>();
8.         Pattern pattern = Pattern.compile("\\d+");
9.         Matcher matcher = pattern.matcher(str);

10.        while (matcher.find()) {
11.            numericParts.add(matcher.group());
12.        }

13.        return numericParts;
14.    }
15. }
```



THANK YOU